

Library Management System

A PROJECT REPORT

Submitted by

Sr No	Name	Registration No
1	Saarthi Patharwal	12405151

*in partial fulfillment for the award of the degree
of*

BACHELOR OF TECHNOLOGY

School of Computer Science and Engineering

Lovely Professional University, Punjab

July 2026

TABLE OF CONTENTS

Sr No	Topic	Page
1	Abstract	3
2	Introduction	4
3	Problem Statement	5
4	Objectives	6
5	Scope	7
6	System Design & Analysis	8
7	Technologies Used	9
8	Architecture Diagram	10
9	Module Description	11
10	Implementation	11
11	Code Snippets	12
12	Screenshots of Program Output	13-14
13	Testing	15
14	Result	16
15	Output Screenshot	16
16	Conclusion	17
17	Future Work	17
18	References	18

ABSTRACT

The Library Management System is a console-based application developed to streamline the day-to-day operations of a small library, including cataloguing books, tracking loans, and managing returns. The primary purpose of this project is to give a librarian or administrator a fast, dependable way to add new titles, search the catalog, issue and return copies, and view the entire collection sorted by title or author, all from a simple text-based menu.

The project is developed entirely in C++, making extensive use of the Standard Template Library (STL) — specifically `std::map` for indexed lookups by ID and ISBN, `std::vector` combined with `std::sort` for on-demand catalog ordering, and `fstream` for automatic, file-based data persistence. Every user input is validated before it is accepted, ensuring the application cannot crash or hang due to unexpected or malformed entries.

By combining clean object-oriented design with efficient STL-backed data structures, this project demonstrates how a lightweight, self-contained inventory system can be built without relying on an external database, network connection, or third-party framework — making it well suited to small libraries, classrooms, or personal book collections.

INTRODUCTION

Managing a book collection by hand — through paper registers or ad-hoc spreadsheets — is slow and error-prone, particularly for a small library with limited staff. A librarian often needs to answer simple but time-sensitive questions: how many copies of a title remain on the shelf, which books have just arrived, and which copies are currently on loan. A lightweight, offline tool that answers these questions reliably, without the overhead of a database server or web infrastructure, offers real practical value.

The Library Management System is a console-based application designed to provide exactly these capabilities. It allows an administrator to add new titles to the catalog, search for existing titles by title, author, or ISBN, issue and return copies, and view the full collection sorted in different ways. All catalog data is automatically written to a local file, so the library's state is preserved between sessions without requiring any manual export or backup step.

The project is implemented entirely in C++, using classes to model individual books and the library itself, and the Standard Template Library to store and organize data efficiently. File handling is performed using the `fstream` library, and every piece of user input is validated to prevent the program from crashing or looping indefinitely on invalid entries. Overall, the project demonstrates how classic data structures and careful software engineering practices can be combined to build a dependable, self-contained inventory management tool.

PROBLEM STATEMENT

Small libraries and personal book collections are often managed using paper registers or basic spreadsheets, making it difficult to reliably track which books are available, which have been borrowed, and how many copies remain in stock. These manual methods are slow, prone to human error, and do not scale well as a collection grows in size.

The Library Management System addresses these problems by providing a lightweight, console-based application that can add, search, issue, and return books through a simple menu-driven interface. The system tracks the total and available quantity of every title, automatically updating these counts as books are issued or returned, and prevents inconsistent states such as issuing a book that has zero copies available or returning more copies than were ever issued. All catalog data is persisted automatically to a local file, ensuring information is never lost between sessions. The project aims to reduce manual effort, eliminate common bookkeeping errors, and give an administrator immediate, accurate visibility into the true state of the collection.

OBJECTIVES

The main objectives of the Library Management System are:

1. To provide a straightforward way to add new books to the catalog, recording title, author, ISBN, and quantity.
2. To allow existing books to be removed from the catalog, either by ISBN or by title.
3. To enable fast searching of the catalog by title, author, or ISBN.
4. To track book availability accurately, updating stock counts automatically whenever a book is issued or returned.
5. To prevent invalid transactions, such as issuing a book with zero copies available or returning more copies than were issued.
6. To provide sortable views of the catalog, ordered by title, author, or most recently added.
7. To persist all catalog data automatically to a local file, so information is retained between sessions.
8. To validate every piece of user input, ensuring the application never crashes or hangs on unexpected input.
9. To demonstrate effective use of the C++ Standard Template Library for a real-world data management task.
10. To build a reliable, dependency-free system that runs entirely offline on a single machine.

SCOPE

The scope of the Library Management System is focused on providing a reliable, console-based tool for managing a small to medium-sized book collection. The system supports adding, deleting, and searching for books, issuing and returning copies, and viewing the catalog sorted by different fields. Because the application relies on local file storage rather than a networked database, it is best suited to single-user environments such as a personal collection, a classroom library, or a small departmental library where concurrent multi-user access is not required.

The project also leaves room for future expansion. Features such as user authentication, automatic fine calculation for overdue books, usage-report generation, and a graphical or web-based front end could all be layered on top of the existing LibraryManager core without altering its fundamental design. With such enhancements, the system could evolve from a single-user console tool into a more feature-rich library management solution suitable for wider use.

SYSTEM DESIGN & ANALYSIS

The Library Management System follows a straightforward two-layer design: a console-based presentation layer and a core logic layer responsible for all business rules. The presentation layer, implemented inside `main()`, is only responsible for displaying menus, collecting and validating user input, and forwarding requests to the core logic layer — it contains no business logic of its own.

The core logic resides in the `LibraryManager` class, which owns the entire in-memory collection of `Book` objects and exposes methods for every supported operation: adding, deleting, searching, issuing, returning, sorting, and persisting the catalog. Internally, `LibraryManager` keeps two STL containers in sync — a `std::map<int, Book>` keyed by a unique, auto-incrementing ID for the primary store, and a `std::map<std::string, int>` that maps each ISBN to its corresponding ID, allowing ISBN-based lookups in logarithmic time without duplicating book records.

When the librarian requests a sorted view of the catalog, `LibraryManager` copies the current books into a `std::vector<Book>` and applies `std::sort` with a custom comparator, since `std::map` cannot be reordered by non-key fields directly. Every change to the catalog — an addition, deletion, issue, or return — is written back to a local text file using `fstream`, ensuring the system's state survives a restart without requiring any manual save step.

TECHNOLOGIES USED

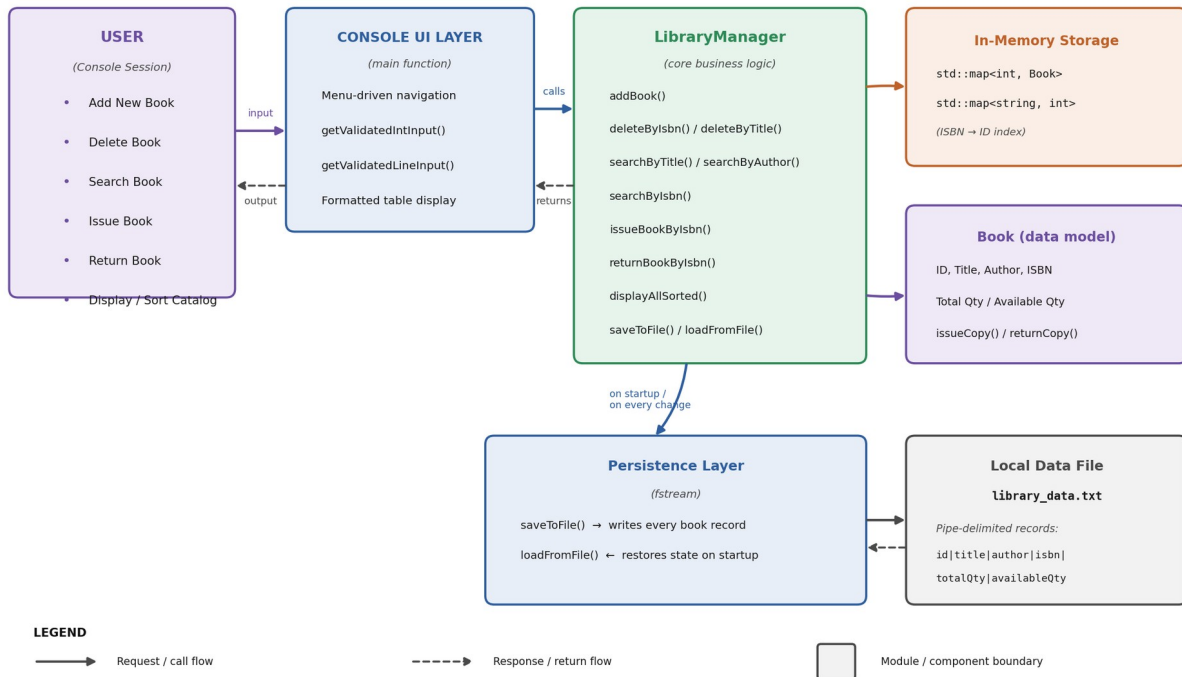
The Library Management System is built using a focused set of C++ technologies, each responsible for a specific part of the application — from modeling data to handling persistent storage.

Technology	Purpose
C++17	Core programming language used to implement the entire system.
Standard Template Library (STL)	Provides <code>std::map</code> , <code>std::vector</code> , and <code>std::sort</code> for efficient data storage, indexing, and ordering.
<code>fstream</code>	Handles reading from and writing to the local data file for persistent storage.
Object-Oriented Programming (OOP)	Used to model <code>Book</code> and <code>LibraryManager</code> as independent, well-encapsulated classes.
<code>iostream</code> / <code>iomanip</code>	Handles all console input and output, and formats the tabular catalog display.
<code>sstream</code>	Serializes and parses individual book records to and from the pipe-delimited file format.
<code>g++</code> / GCC (C++17 standard)	Compiler used to build and test the application (<code>g++ -std=c++17</code>).

This combination of tools keeps the application entirely self-contained: no external database, network service, or third-party library is required to build, run, or persist data for the system.

ARCHITECTURE DIAGRAM

Library Management System Architecture Diagram



The diagram above illustrates the flow of a request from the console user through the UI layer into LibraryManager, which coordinates the in-memory Book records and ISBN index before persisting every change to the local data file via fstream.

MODULE DESCRIPTION

The Library Management System consists of the following major modules:

- 1. Console UI Module (main()):** Displays the menu, validates every user input, and calls into LibraryManager to fulfill each request.
- 2. Book Module (Book class):** Represents a single book's data — ID, title, author, ISBN, and quantities — along with the logic to issue or return one copy.
- 3. LibraryManager Module:** Owns the catalog and provides every operation for adding, deleting, searching, issuing, returning, and sorting books.
- 4. Persistence Module:** Serializes and deserializes book records to and from a local text file using fstream, keeping the catalog durable across sessions.
- 5. Input Validation Module:** Ensures every prompt — menu choice, quantity, or text field — is checked before being accepted, preventing crashes or infinite loops on bad input.

IMPLEMENTATION

The Library Management System is implemented as a single C++ source file, structured around two core classes — Book and LibraryManager — supported by a thin console UI layer in main(). The Book class encapsulates a book's fields and the logic to issue or return a single copy, while LibraryManager owns a `std::map<int, Book>` for the master catalog and a `std::map<std::string, int>` ISBN index for fast lookups. All persistence is handled through fstream, reading from and writing to a local file named `library_data.txt` in a simple pipe-delimited format. Input validation is centralized in reusable helper functions that repeatedly re-prompt the user until valid data is entered, guarding against both non-numeric input and out-of-range menu choices. The application was compiled and tested using g++ with the `-std=c++17` flag.

CODE SNIPPETS

[Code snippet to be inserted here]

[Code snippet to be inserted here]

SCREENSHOTS OF PROGRAM OUTPUT

1. Main Menu

[Screenshot to be inserted here]

2. Add / Search Book

[Screenshot to be inserted here]

3. Issue / Return Book

[Screenshot to be inserted here]

4. Sorted Catalog Display

[Screenshot to be inserted here]

TESTING

The Library Management System was tested to ensure that all major features function correctly and provide a smooth, reliable user experience. Functional testing was performed on every core module, and the results confirmed that the system behaved as expected in each case.

Test Case	Expected Result	Status
Add Book	New book is added to the catalog successfully	Passed
Delete Book (by ISBN)	Matching book is removed from the catalog	Passed
Delete Book (by Title)	Book is removed and duplicate titles are reported	Passed
Search by Title / Author / ISBN	Correct matching book(s) are returned	Passed
Issue Book	Available quantity decreases correctly	Passed
Issue Book (no copies left)	Issue is blocked with a clear message	Passed
Return Book	Available quantity increases correctly	Passed
Return Book (already full stock)	Return is blocked with a clear message	Passed
Sort by Title / Author	Catalog is displayed in the correct order	Passed
Invalid Menu Input	Program re-prompts without crashing	Passed
Data Persistence	Catalog is restored correctly after restart	Passed

RESULT

The Library Management System was successfully developed, compiled, and tested. All core modules performed as expected, including adding, deleting, searching, issuing, returning, and sorting books, as well as saving and loading data from the local file. The application handled invalid input gracefully in every test case, and the catalog's state was preserved correctly across multiple program restarts. These results confirm that the system meets its design goals of providing a reliable, dependency-free tool for managing a small library's book inventory.

OUTPUT SCREENSHOT

Sample Catalog Display

[Output screenshot to be inserted here]

CONCLUSION

The Library Management System was developed to provide a simple, dependable, console-based solution for managing a small library's book inventory. By combining an object-oriented design with the C++ Standard Template Library, the project demonstrates how `std::map`, `std::vector`, `std::sort`, and `fstream` can be used together to build an efficient, self-contained inventory system. Robust input validation and automatic file-based persistence ensure the application remains stable and useful in everyday operation. Overall, the project shows how classic C++ programming techniques can be applied to solve a practical, real-world problem without relying on external frameworks or databases.

FUTURE WORK

While the current system fulfills its core requirements, several enhancements could extend its capabilities. User authentication and role-based access could be introduced to support multiple librarians with different permission levels. A fine-calculation module could track overdue books and compute penalties automatically. The system could also be extended with a graphical or web-based front end, or migrated to a proper database engine to support concurrent multi-user access. Generating usage reports — such as most-borrowed titles or overdue summaries — would further increase the system's value for real-world library administration.

REFERENCES

1. cppreference.com. C++ Standard Library Reference. Available at: <https://en.cppreference.com/>
2. cppreference.com. std::map Container Reference. Available at: <https://en.cppreference.com/w/cpp/container/map>
3. cppreference.com. std::sort Algorithm Reference. Available at: <https://en.cppreference.com/w/cpp/algorithm/sort>
4. cppreference.com. <fstream> Header Reference. Available at: <https://en.cppreference.com/w/cpp/header/fstream>
5. cplusplus.com. C++ Language Reference. Available at: <https://cplusplus.com/reference/>
6. GNU Project. GCC, the GNU Compiler Collection Documentation. Available at: <https://gcc.gnu.org/onlinedocs/>
7. ISO/IEC. International Standard ISO/IEC 14882 — Programming Languages — C++. Available at: <https://www.iso.org/>
8. Stroustrup, B. The C++ Programming Language (4th Edition). Addison-Wesley, 2013.
9. GeeksforGeeks. Library Management System in C++. Available at: <https://www.geeksforgeeks.org/>
10. GitHub Documentation. Available at: <https://docs.github.com/>